

Predicting Market Outcomes from Earnings Call Transcripts: Fine-tuning, Retrieval, and Prompting

Surya Suresh

Nikhil Kumar

Dheeraj Gosula

Abstract

We study whether a modern open-weight large language model can predict financial outcomes from S&P 500 earnings call transcripts, and which adaptation techniques actually help. We test the model on two binary classification tasks: stock direction (whether the price moves up or down over the three trading days after the call) and earnings surprise (whether reported EPS beats analyst consensus). For each task we evaluate three techniques: LoRA fine-tuning, retrieval-augmented generation (RAG), and few-shot prompting. Using Qwen2.5-7B-Instruct as a fixed base model, we run every combination of these techniques on both tasks and report results across 16 conditions. LoRA fine-tuning produces a substantial improvement on the surprise task (AUC 0.637 to 0.812, 95% CI excluding random) but no measurable improvement on stock direction (AUC remains near 0.50). Few-shot prompting and retrieval show smaller, mixed effects: they tend to help the unadapted base model but slightly hurt the fine-tuned one. The asymmetry between the two tasks suggests that LLMs fine-tuned on earnings calls can identify outcomes already discussed in the transcript, but that predicting stock movement from text alone remains close to chance.

1 Introduction

Earnings calls are quarterly events in which a public company’s management reports financial results and answers questions from sell-side analysts. The accompanying transcripts are dense, structured documents that contain explicit financial figures, forward-looking guidance, and unscripted exchanges in the question-and-answer (Q&A) section. They are widely read by investors, but extracting predictive signal from them is hard: short-horizon stock returns are noisy, and the most informative content is mixed with boilerplate disclosures and scripted pleasantries.

Most prior work on earnings call NLP relies on sentiment scoring or single-task classifiers (Loughran and McDonald, 2011; Araci, 2019; Qin and Yang, 2019). More recent work has applied larger transformer architectures (Koval et al., 2023; Ma et al., 2020), but no study has systematically compared the three techniques most commonly used to adapt large language models to a target domain—supervised fine-tuning, retrieval-augmented generation (RAG) (Lewis et al., 2020), and few-shot prompting (Brown et al., 2020; Wei et al., 2022)—on the same transcripts and the same prediction tasks.

We close this gap with a 2^3 factorial experiment. For each of two tasks (stock direction and earnings surprise) we toggle fine-tuning, RAG, and few-shot prompting independently and run all eight resulting combinations. We use Qwen2.5-7B-Instruct (Qwen Team, 2024) as a fixed base model and adapt it efficiently with QLoRA (Detrmers et al., 2023; Hu et al., 2022), allowing the entire pipeline to run on a single 16 GB V100 GPU.

Our contributions are:

- A controlled comparison of fine-tuning, retrieval, and prompting for transcript-based outcome prediction, with bootstrap confidence intervals on every reported metric.
- Evidence that LoRA fine-tuning yields a large, statistically meaningful gain on EPS surprise (+0.18 AUC over zero-shot base) while leaving stock direction at chance.
- A head-and-tail truncation strategy that fits within the V100 context budget while preserving both opening remarks and the Q&A section, two parts of the call that prior work flags as high-signal (Qin and Yang, 2019; Cohen et al., 2020).
- Analysis of the asymmetry between the two tasks: surprise is largely answerable from in-document linguistic and numerical cues, whereas three-day

stock direction depends on factors outside the transcript.

2 Related Work

Sentiment-based approaches. The earliest text-based methods for financial prediction use sentiment dictionaries such as Loughran-McDonald (Loughran and McDonald, 2011) or domain-specific classifiers like FinBERT (Araci, 2019; Yang et al., 2020). These models compress an entire transcript into a small number of sentiment scores, which are fed to a downstream classifier. The compression discards structural information, and as we confirm in Section 6, reduces stock direction prediction to roughly chance accuracy.

Earnings call deep learning. Qin and Yang (2019) use multimodal text-and-audio features to predict post-call return volatility. Ma et al. (2020) combine GloVe embeddings with industry-sector signals to predict next-day stock movement and report 48.5–56.8% accuracy across sectors. Koval et al. (2023) apply hierarchical FinBERT to earnings surprise classification and report 76.56% accuracy on a balanced test set. Cohen et al. (2020) show that the unscripted Q&A portion of calls carries more predictive signal than the scripted prepared remarks. Each of these works tests a single architecture on a single task; none compare across modern adaptation techniques.

Domain LLMs and parameter-efficient adaptation. BloombergGPT (Wu et al., 2023) pretrains a 50B-parameter model on Bloomberg’s proprietary financial corpus. We instead use a smaller, openly available model (Qwen2.5-7B) and adapt it with LoRA (Hu et al., 2022) and QLoRA (Dettmers et al., 2023), which combines 4-bit weight quantization with low-rank trainable adapters and fits fine-tuning on a single 16 GB GPU. Generic adaptation techniques—retrieval (Lewis et al., 2020) and chain-of-thought prompting (Wei et al., 2022)—are well studied in open-domain NLP but, to our knowledge, untested side-by-side on earnings call outcome prediction.

3 Data

We use the publicly available S&P 500 earnings transcripts corpus `Bose345/sp500_earnings_transcripts` on HuggingFace, which contains tens of thousands of calls. From this corpus we

Task	Total	Train	Test	% pos.
Direction	1,965	1,768	197	51.1
Surprise	827	744	83	80.9

Table 1: Dataset sizes after fetching labels. Train/test split is chronological 90/10 (no shuffle). % pos. is the fraction labeled *up* (direction) or *beat* (surprise).

sample the most recent 5,000 transcripts (collected May 2024 – May 2025) and attempt to fetch labels for each via the `yfinance` Python library. Coverage differs by label type. For stock direction, we fetch the closing price on the last trading day before the call and on the third trading day after; the label is 1 (up) if the post-call price exceeds the pre-call price, 0 (down) otherwise. 1,965 calls yield valid direction labels. For earnings surprise, we use `yfinance`’s `get_earnings_dates()` method to retrieve reported EPS and analyst consensus, requiring a match within 10 days of the call; the label is 1 (beat) if reported $\geq$ consensus, 0 (miss) otherwise. 827 calls yield valid surprise labels.

The sharply lower coverage for surprise reflects `yfinance`’s limited EPS data for delisted, recently-IPO’d, or non-standard tickers (class-B dual-share entries, recent corporate actions, etc.), and is the single largest external constraint on our experimental scale. Table 1 summarizes the resulting datasets. Direction is nearly class-balanced; surprise is heavily skewed toward beat—reflecting the well-known tendency of S&P 500 firms to beat consensus more often than they miss. We mitigate this during training (Section 4.2) but preserve the natural distribution at test time so that reported metrics reflect deployment conditions.

For each task we sort calls by date and use the earliest 90% for training and the latest 10% for testing. A random split would leak information from later calls into earlier predictions and inflate test performance, particularly for direction where the macro market regime itself carries non-trivial signal. Chronological splitting prevents this look-ahead bias and matches the deployment setting in which a model trained on past calls must predict on new ones.

Transcripts are long: the median length under the Qwen tokenizer is approximately 13,000 tokens, far exceeding our 2,048-token budget per training example. Naive front-truncation (keep first 2,048 tokens) preserves the prepared opening remarks but discards the entire Q&A section. The Q&A is unscripted, contains analyst pressure on specific

concerns, and is widely cited as a high-signal segment of the call (Qin and Yang, 2019; Cohen et al., 2020). We instead use a head-and-tail truncation: keep the first 60% (1,229 tokens) and the last 40% (819 tokens) of the budget, dropping only the middle. This preserves both the management narrative and the Q&A. The same truncation is used at training and at evaluation so that the prompt distribution seen by the model is consistent across the two phases.

4 Methodology

4.1 Task formulation

Each prompt is constructed as

<system instruction>

Transcript:

<truncated transcript>

Answer:

where <system instruction> is a one-paragraph description of the task with the allowed answer tokens (*up/down* or *beat/miss*). The model is asked to emit one token after Answer:.

At inference we run a single forward pass and read the logits at the final position. We extract the two target token IDs (e.g., `_up` and `_down`) and apply softmax over just that pair to obtain a probability of the positive class. This avoids generation entirely: we never sample from the model and never parse free-form text. The same scoring procedure is used for the zero-shot base model and for all augmented variants, so all conditions are directly comparable.

4.2 Fine-tuning with QLoRA

We fine-tune Qwen2.5-7B-Instruct (Qwen Team, 2024) using QLoRA (Dettmers et al., 2023), which combines 4-bit weight quantization with low-rank adapter training (Hu et al., 2022). The base model is loaded with 4-bit NF4 quantization and double quantization, reducing weight memory from ~ 14 GB (fp16) to ~ 4 GB—enough headroom to fit the 7B model on a 16 GB V100 along with activations and gradients. We attach LoRA adapters to the four attention projection matrices in every transformer block (`q_proj`, `k_proj`, `v_proj`, `o_proj`) at rank $r = 16$, scaling $\alpha = 32$, and dropout 0.05. Feed-forward and embedding layers are entirely frozen, yielding approximately 10 M trainable parameters out of 7.6 B total (0.13%). Targeting atten-

tion projections is the standard recipe in the LoRA literature: attention controls how tokens combine information across positions, which is the locus of any change we want a domain-adapted model to learn (e.g., attending more strongly to Q&A guidance language).

We train each task’s adapter independently for 3 epochs with the AdamW optimizer in 8-bit paged form. Learning rate is $2e-4$ with cosine decay and 3% linear warmup, per-device batch size is 1 with gradient accumulation over 8 steps for an effective batch size of 8, and mixed-precision is fp16 (V100 hardware does not support bf16). We enable gradient checkpointing to fit activations within memory and apply a label mask so that the cross-entropy loss is computed only on the answer token. For surprise (81% beat / 19% miss) the natural class skew causes the model to collapse onto the majority class, so we oversample the minority class within the training split only by sampling miss examples with replacement until both classes have equal counts; the validation and test splits are left at their natural distribution. We evaluate on the chronological validation set after each epoch and retain the checkpoint with lowest validation loss, which guards against the onset of overfitting we observe after epoch 1 on the smaller surprise dataset.

Each fine-tuned model is saved as a ~ 40 MB LoRA adapter alongside the saved tokenizer, decoupled from the unchanged base weights. Adapters are applied at inference by loading the base model with the same 4-bit quantization and attaching the adapter via the PEFT library; the same procedure is used for the RAG and few-shot variants that build on top of the fine-tuned model.

4.3 Few-shot prompting

[Section to be completed by teammate.] The few-shot variant prepends k labeled examples drawn chronologically from the training split before the test transcript. Per the team’s design, $k = 4$ shots are used, balanced across labels (2 positive, 2 negative), with 150 tokens per shot and a 3072-token total prompt budget. Chain-of-thought reasoning was tested but did not yield reliable improvements in our setting and is excluded from the final pipeline, likely because the additional reasoning step disturbs the single-token answer format the adapter was trained on. Extractive sentence selection using a 24-keyword financial dictionary is used to densify each shot’s transcript content. Final scoring uses the same softmax-over-target-pair procedure

described in Section 4.1.

4.4 Retrieval-augmented generation

[Section to be completed by teammate.] The retrieval corpus contains ~ 450 documents, one per training transcript, each consisting of the company ticker, call date, and first 2,000 characters of the transcript. Retrieval is sparse: the system first attempts a same-company prior-quarter lookup (most recent transcripts from the same ticker with date strictly preceding the query call), then falls back to BM25 keyword ranking over the full training corpus to fill remaining slots. Retrieved passages are prepended to the prompt within a ~ 400 -token budget. The corpus is restricted to the training split to prevent any test leakage.

5 Experimental Setup

All training and evaluation runs use a single NVIDIA V100 PCIE 16 GB GPU on the OSC Pitzer cluster, with the Python environment pinned to PyTorch 2.3.1+cu118 (the latest version supporting the V100’s sm_70 architecture), CUDA 11.8, transformers 4.44.2, peft 0.13.2, and bitsandbytes 0.43.3.

For each task we evaluate eight conditions formed by toggling fine-tuning, RAG, and few-shot prompting: zero-shot base, fine-tuned only, RAG only, prompting only, and the four pairwise and triple combinations. The base-only and prompting-only variants share a single set of base-model weights; the fine-tuned variants use a task-specific adapter on top of the same base model.

For each condition we report three metrics. The plain prediction-vs-label match at threshold 0.5 (accuracy) is reported but not relied on for claims, since on imbalanced tasks it is dominated by the class prior. Balanced accuracy is the mean of per-class recall, $(\text{recall}_+ + \text{recall}_-)/2$, which is robust to class imbalance. AUC-ROC with a 95% bootstrap confidence interval (1,000 resamples) is our primary metric: it is threshold-free, robust to imbalance, and the CI quantifies how much of the gap to chance is statistically real given the test-set size. We treat a result as above chance only if its 95% AUC CI excludes 0.50, and treat one model as better than another only when their CIs are clearly separated; otherwise we report the gap with the caveat that it is within sampling noise.

Task	Model	Acc	B-Acc	AUC	95% CI
Direction	Base	0.569	0.502	0.493	[0.41, 0.57]
Direction	Fine-tuned	0.594	0.500	0.499	[0.42, 0.58]
Surprise	Base	0.759	0.566	0.637	[0.49, 0.78]
Surprise	FT	0.807	0.708	0.812	[0.69, 0.92]

Table 2: Fine-tuning vs. zero-shot base. Direction $n = 197$, surprise $n = 83$. *B-Acc* is balanced accuracy. AUC CIs are bootstrap, 1,000 resamples.

6 Results

6.1 Fine-tuning vs. zero-shot base

Table 2 presents the headline fine-tuning results.

On the surprise task, fine-tuning improves AUC by 0.18 ($0.637 \rightarrow 0.812$) and balanced accuracy by 0.14 ($0.566 \rightarrow 0.708$). The fine-tuned AUC’s 95% CI $[0.69, 0.92]$ cleanly excludes random chance (0.50). The two CIs overlap by a small margin in the $[0.69, 0.78]$ window, so the base-vs-fine-tuned gap is best described as substantial and likely real, but at the edge of statistical separation given $n = 83$.

On the direction task, both base and fine-tuned models hover near AUC 0.50 with overlapping CIs. The fine-tuned model’s plain accuracy (0.594) matches the majority-class baseline ($117/197 \approx 0.594$) almost exactly, indicating the model has effectively learned to predict “up” on every example. Balanced accuracy is exactly 0.500, confirming this collapse. Despite minority-class oversampling during training, the model could not extract any directional signal beyond the prior.

These results are broadly consistent with prior work. Ma et al. (2020) report 48.5–56.8% accuracy across sectors for next-day stock movement using GloVe and industry features—similar to our chance-level direction result, despite their use of explicit sector signals. Koval et al. (2023) report 76.56% accuracy on a balanced earnings surprise test set with hierarchical FinBERT; our fine-tuned model reaches 80.7% accuracy on a 77/19 imbalanced test and AUC 0.812 on the same test, suggesting that decoder-LLM fine-tuning is at least competitive with task-specialized encoder architectures on this task. We do not claim a head-to-head improvement since the test sets and label-fetching protocols differ.

6.2 Few-shot prompting

[Analysis to be completed by teammate.] The 4-shot prompting results from our presentation slides

are summarized in Table 3 for completeness; the discussion of *why* the effect of prompting is asymmetric across base and fine-tuned models, and across the two tasks, is left to the teammate who designed the prompting pipeline.

Condition	Zero-shot AUC	4-shot AUC
Base direction	0.493	0.520
FT direction	0.499	0.536
Base surprise	0.637	0.720
FT surprise	0.812	0.748

Table 3: Few-shot prompting AUC, copied from team presentation. Confidence intervals are reported in the original slide deck.

6.3 Retrieval-augmented generation

[Analysis to be completed by teammate.] Table 4 summarizes the RAG results from the team presentation; analysis of why RAG helps the base model on surprise but slightly hurts the fine-tuned model is left to the teammate.

Cond.	ZS	4-shot	RAG	RAG+4
Base direction	0.493	0.520	0.492	0.532
FT direction	0.499	0.536	0.494	0.538
Base surprise	0.637	0.720	0.680	0.676
FT surprise	0.812	0.748	0.752	0.722

Table 4: Combined results across all four conditions. ZS = zero-shot. Numbers are AUC; full CIs are reported in the original slide deck.

7 Discussion

7.1 Why does fine-tuning help one task but not the other?

The asymmetry between surprise and direction in Table 2 is not a quirk of model capacity or training procedure—it reflects a fundamental difference in where the predictive signal lives.

For surprise, the relevant information is largely contained in the transcript itself. The earnings press release that announces actual EPS is published roughly 30 minutes before the call begins; by the time the CFO reads opening remarks, the beat or miss is already public, and management almost always references the figures explicitly (e.g., “We delivered adjusted EPS of \$1.85, ahead of consensus of \$1.77”). Even when our truncation drops the precise numbers, the framing, tone, guidance language, and Q&A dynamics provide strong prox-

ies. Fine-tuning teaches the model which proxies to attend to.

For direction, the label is determined by post-call market behavior over three trading days—a function of how thousands of investors interpret the call and of intervening macro news, rate moves, sector rotations, and quarterly fund flows. None of that is in the transcript. The model is being asked to forecast something whose causal driver is largely outside the input. We should expect text-only models to fail at this, and they do.

This is consistent with the broader literature on post-earnings announcement drift: even with audio features and structured market data, prior work reports only modest gains over chance on short-horizon direction (Qin and Yang, 2019; Sawhney et al., 2020), and our text-only setup is strictly less informed.

7.2 The role of head-and-tail truncation

Several earnings call NLP studies note that the Q&A section carries more candid signal than the prepared remarks (Qin and Yang, 2019; Cohen et al., 2020), since management’s scripted opening tends to be uniformly positive regardless of actual performance. Our head-and-tail strategy operationalizes this: at the same 2,048-token budget, we retain both segments rather than discarding the Q&A entirely. We did not run a controlled ablation against front-truncation, but our 0.812 AUC is at the high end of comparable BERT-family results on similar transcript spans, suggesting the truncation choice is consequential. A natural follow-up experiment is to repeat the surprise fine-tune with strict front-truncation under matched conditions and quantify the gap.

7.3 Class imbalance and metric choice

The surprise task’s 81/19 class skew is a recurring trap. A model that predicts “beat” on every example achieves $\sim 77\%$ accuracy with zero predictive skill, which is dangerously close to our base model’s 0.759 accuracy. We rely on AUC and balanced accuracy precisely because they cannot be inflated by majority-class collapse. The direction task illustrates the same point in reverse: the fine-tuned model’s plain accuracy (0.594) looks like a small improvement over base (0.569), but its balanced accuracy (0.500) reveals that all gain comes from learning the prior more confidently, not from any new discrimination ability.

7.4 Effect of method combination

Reading across Tables 2–4, two patterns emerge. First, on the surprise task, every additional augmentation on top of fine-tuning slightly degrades AUC ($0.812 \rightarrow 0.748$ with 4-shot; $0.812 \rightarrow 0.752$ with RAG; $0.812 \rightarrow 0.722$ with both). The fine-tuned adapter has already learned what to look for from the in-document signal, and adding noisier context appears to crowd out the signal-rich Q&A within the fixed context window. Second, on the surprise task without fine-tuning, augmentations help (4-shot: $0.637 \rightarrow 0.720$; RAG: $0.637 \rightarrow 0.680$). For an unadapted base model more context is better; for an adapted one, the context budget is better spent on the test transcript itself. This is consistent with prior findings that retrieval and in-context learning provide diminishing returns once a task has been fine-tuned.

For direction, no condition meaningfully exceeds chance, suggesting the bottleneck is signal availability in the input, not modeling choices.

8 Limitations

Temporal coverage. Our entire dataset spans roughly one year (May 2024 – May 2025). This covers a single market regime and only a few earnings cycles. Performance under different macro conditions (e.g., a correction, a high-volatility episode, or rate-cycle inflection) is unknown. Expanding to multi-year data would improve external validity and would likely tighten the surprise CIs by enlarging the test set.

Test set size. The surprise test set ($n = 83$, of which 19 are *miss*) is small enough that AUC confidence intervals are wide (± 0.11). This is driven by yfinance’s limited EPS coverage—the analyst-consensus endpoint succeeds on only $\sim 40\%$ of S&P 500 tickers in our sampled window. A different EPS data source (FactSet, Refinitiv, yahoo-query) would likely double the labeled dataset.

Single base model. We did not run an ablation across base LLMs. Llama-3-8B, Gemma-2-9B, and finance-pretrained models like FinMA may behave differently. The qualitative conclusion—that fine-tuning helps surprise but not direction—would likely hold (the signal-availability story is model-agnostic), but specific AUC numbers would shift.

Truncation budget. Our 2,048-token cap is a hardware constraint of the V100. Extending to

4096 or 8192 tokens would let us include more middle prepared remarks and more Q&A content; this is a natural follow-up on an A100 or larger GPU.

No tradeable edge. Surprise prediction from transcripts is a useful capability test, but not a tradeable signal. By the time the call begins, the press release with actual EPS numbers has been public for ~ 30 minutes; the “prediction” is in fact a reading-comprehension task. We frame our results accordingly: the surprise gain is evidence of financial language understanding, not of forecasting capability.

9 Conclusion

We tested whether a modern open-weight LLM, adapted via fine-tuning, retrieval, or prompting, can predict financial outcomes from earnings call transcripts. LoRA fine-tuning produces a clear and statistically meaningful gain on EPS surprise (AUC 0.812 vs. 0.637 base) but does not move stock direction prediction off chance. The asymmetry is interpretable: surprise is largely answerable from in-document evidence, while three-day stock direction depends on factors absent from the transcript. Combining augmentations on top of a fine-tuned adapter provides no further gain on surprise and often slightly hurts. Our results support the use of fine-tuned LLMs for transcript-grounded financial classification tasks, and caution against expecting them to forecast short-horizon market reactions from text alone.

Author Contributions

Surya Suresh.

Nikhil Kumar.

Dheeraj Gosula.

Acknowledgements

AI assistants (Claude and ChatGPT) were used to help organize and format the LaTeX code for this report and as a mild writing assistant.

References

Dogu Araci. 2019. FinBERT: Financial Sentiment Analysis with Pre-trained Language Models. *arXiv preprint arXiv:1908.10063*.

- Tom Brown, Benjamin Mann, Nick Ryder, et al. 2020. Language Models are Few-Shot Learners. In *NeurIPS*.
- Lauren Cohen, Christopher Malloy, and Quoc Nguyen. 2020. Lazy Prices. *Journal of Finance*, 75(3):1371–1415.
- Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. QLoRA: Efficient Finetuning of Quantized LLMs. In *NeurIPS*.
- Edward Hu, Yelong Shen, Phillip Wallis, et al. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *ICLR*.
- Ross Koval, Nicholas Andrews, and Xifeng Yan. 2023. Forecasting Earnings Surprises from Conference Call Transcripts. In *Findings of ACL*.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, et al. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *NeurIPS*.
- Tim Loughran and Bill McDonald. 2011. When Is a Liability Not a Liability? Textual Analysis, Dictionaries, and 10-Ks. *Journal of Finance*, 66(1):35–65.
- Yu Ma, Rui Mao, Qika Lin, et al. 2020. Multi-Source Aggregated Classification for Stock Price Movement Prediction. *Information Fusion*.
- Yao Qin and Yi Yang. 2019. What You Say and How You Say It Matters: Predicting Stock Volatility Using Verbal and Vocal Cues. In *ACL*, pages 390–401.
- Qwen Team. 2024. Qwen2.5 Technical Report. *arXiv preprint arXiv:2412.15115*.
- Ramit Sawhney, Piyush Khanna, Arshiya Aggarwal, Taru Jain, Puneet Mathur, and Rajiv Ratn Shah. 2020. VolTAGE: Volatility Forecasting via Text Audio Fusion with Graph Convolution Networks for Earnings Calls. In *EMNLP*.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, et al. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *NeurIPS*.
- Shijie Wu, Ozan Irsoy, Steven Lu, et al. 2023. BloombergGPT: A Large Language Model for Finance. *arXiv preprint arXiv:2303.17564*.
- Yi Yang, Mark Christopher Siy Uy, and Allen Huang. 2020. FinBERT: A Pretrained Language Model for Financial Communications. *arXiv preprint arXiv:2006.08097*.